

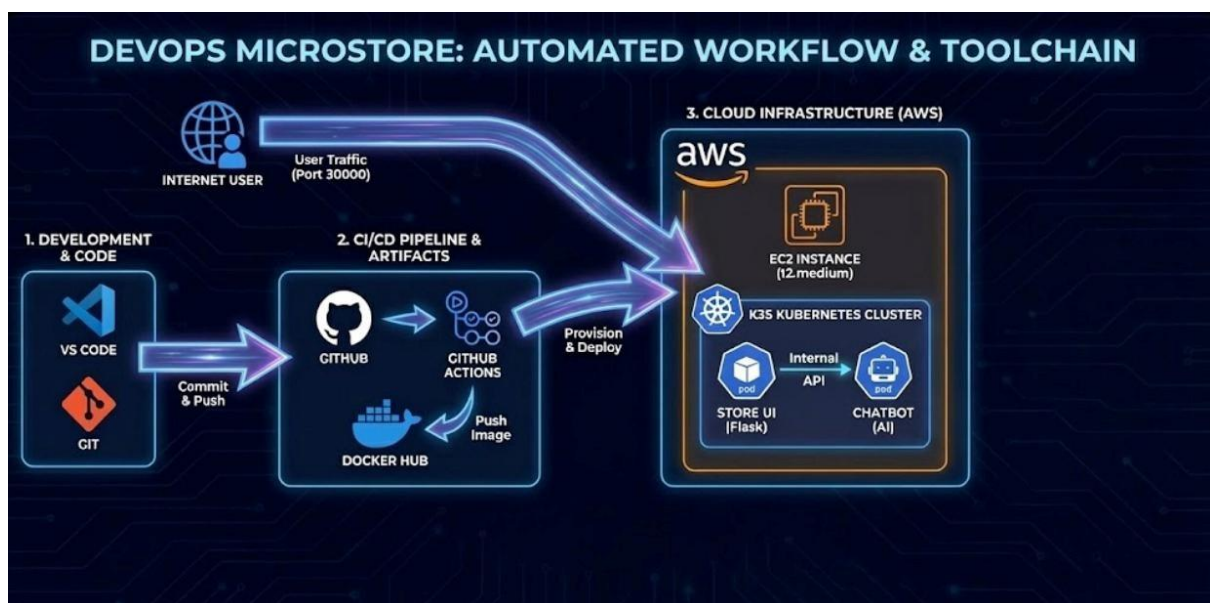
Technical Project Report:

Automated Microservices Deployment

1. Introduction

This project establishes a high-availability, automated infrastructure for a dual-service application

(Chatbot and Store). It leverages the "**Infrastructure as Code (IaC)**" philosophy and **Continuous Integration/Continuous Deployment (CI/CD)** to ensure that software delivery is fast, secure, and repeatable.



2. Toolset & Technologies

Tool	Core Responsibility	Why it was chosen?
HashiCorp Terraform	Infrastructure Provisioning	Provides a declarative way to build and version cloud hardware.
Docker	Containerization	Packages the apps into "units" that run exactly the same everywhere.
Kubernetes (K8s)	Container Orchestration	Manages scaling, self-healing, and networking for the live containers.

Tool	Core Responsibility	Why it was chosen?
GitHub Actions	CI/CD Pipeline	Native integration with the code repository to automate build and deploy steps.



3. Detailed File Breakdown

A. Configuration & Infrastructure

- **main.tf (Terraform):** The blueprint. It defines the "Physical" layer (Servers, VPCs). Without this, there is no place to host the apps.
- **.gitignore:** The security filter. It ensures that sensitive files like **SSH Private Keys (y)**, **Terraform State**, and **.env** files are never exposed in the repository.

B. Application & Local Development

- **Dockerfile:** The build recipe. It converts Python code into a Docker Image by installing dependencies listed in `requirements.txt`.
- **docker-compose.yaml:** The local orchestrator. It allows a developer to run the entire system on a single laptop using a **Bridge Network (ecommerce-net)**. It handles the `depends_on` logic, ensuring the Chatbot is ready before the Store starts.

C. Deployment & Orchestration

- **deploy.yml (GitHub Actions):** The automation engine. It triggers on every push. It uses **GitHub Secrets** to securely handle the `SSH_PRIVATE_KEY` for server access and Docker Hub credentials.
- **deployment.yml (Kubernetes):** The production manager. It defines:
 - **Deployments:** Manages the Pods.
 - **Services:** Maps internal app ports (5000/5001) to the external **Port 30000** via NodePort.
 - **imagePullPolicy: Always:** An essential instruction that forces Kubernetes to pull the newest version of your image from Docker Hub immediately.

4. The DevOps Workflow (Stage by Stage)

1. **Code Change:** A developer updates the Store app (e.g., changing a price).
2. **Commit & Push:** The code is sent to GitHub.
3. **CI Build (deploy.yml):** GitHub Actions builds a new Docker Image and pushes it to **Docker Hub** (harshshahdeo/).
4. **CD Deployment:** GitHub Actions uses SSH to tell the production cluster to update.
5. **Orchestration:** Kubernetes sees the update, pulls the image (because of `imagePullPolicy: Always`), and restarts the Pods.
6. **External Access:** The user sees the new price at `http://<IP>:30000`.

5. Conclusion

By integrating these tools, the project achieves **zero-touch deployment**. The infrastructure is handled by Terraform, the apps are isolated by Docker, and the lifecycle is managed by GitHub Actions and Kubernetes. This architecture is industry-standard for scaling modern web applications.